Search Trees

A search tree is a multi-way tree that is used to guide the search for a record, given the value of one of its fields.

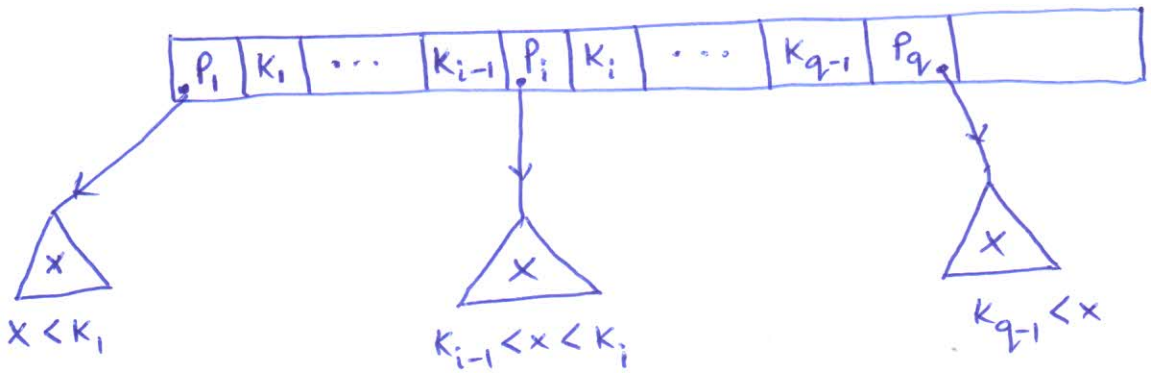
A search key of oder p is a tree such that each node contains <u>at most</u> $p-1$ search values and p pointers in the order

$$< P_1, K_1, P_2, K_2, \cdots, P_{q-1}, K_{q-1}, P_q >$$

where $q \leq p$, each P_i is a pointer to a child node (or a null pointer), and each K_i is a search value from some <u>ordered set of values</u>.

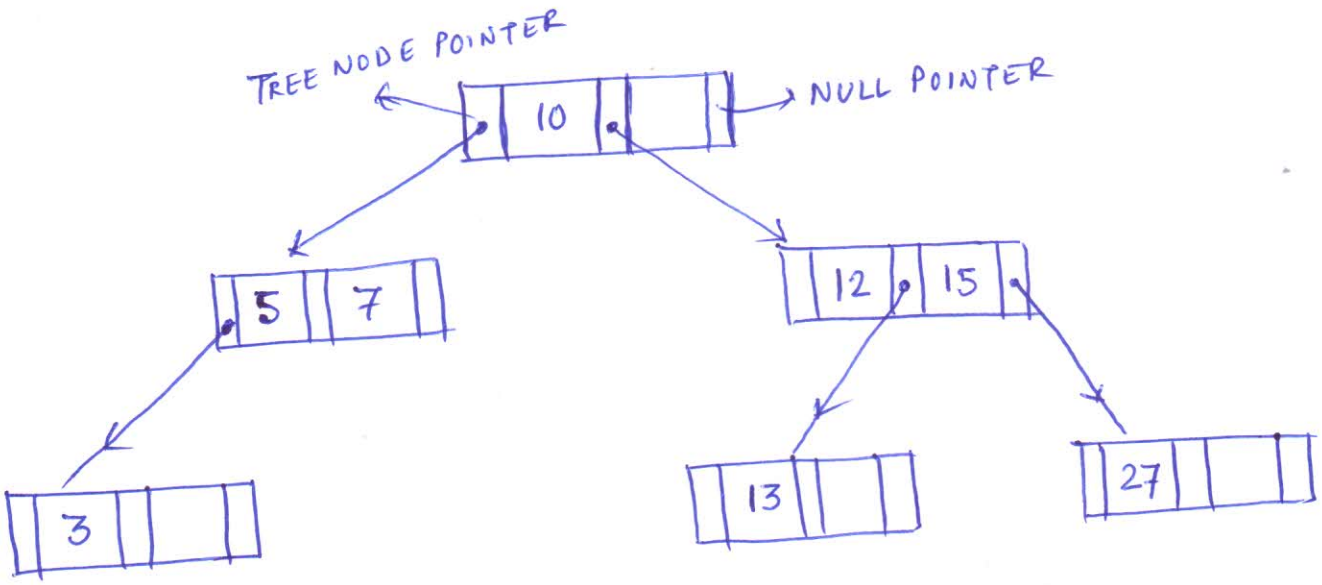
Properties

1. Within each node, $K_1 < K_2 < \cdots < K_{q-1}$
2. For all values X in the subtree pointed at by P_i, we have $K_{i-1} < X < K_i$ for $1 < i < q$, $X < K_i$ for $i = 1$, and $K_{i-1} < X$ for $i = q$



A node in a Search Tree

A Search Tree of order $p = 3$



Note: (1) Insertion and deletion operations in a search tree do not guarantee that the resultant tree is balanced.

(2) Record deletion may leave some nodes in the tree nearly empty, thus wasting storage space and increasing the no. of levels.

B-Trees (Named as Bayer, Boeing, Balanced..., but not Binary)

B-tree is a search tree with some additional constraints that solves the balanced issues of a search tree to some extent.

A B-tree of order p can be defined as

(1) Each internal node in the B-tree is of the form

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \ldots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle$$

where $q \le p$. Each P_i is a tree pointer, a pointer to another node in the B-tree.
Each Pr_i is a data pointer (either a block address or a record address), a pointer to the record whose search key field value is equal to K_i (or to the data block containing that record).

(2) Within each node, $K_1 < K_2 < \cdots < K_{q-1}$

(3) For all search key field values X in the subtree pointed at by P_i, we have $K_{i-1} < X < K_i$ for $1 < i < q$, $X < K_i$ for $i = 1$ and $K_{i-1} < X$ for $i = q$.

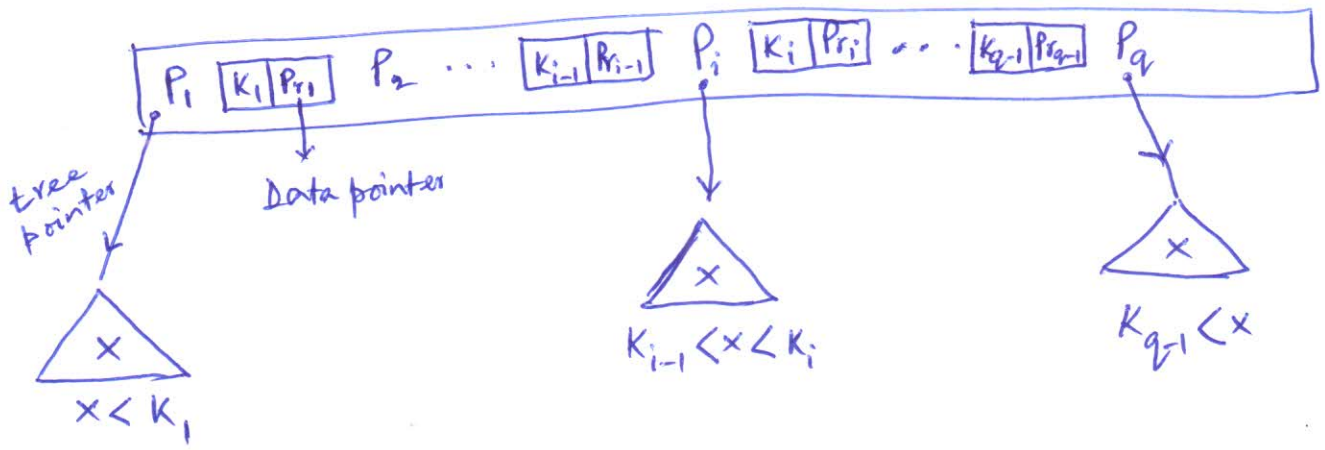
(4) Each node has at most p tree pointers.

(5) Each node, except the root and leaf nodes, has at least $\lceil (p/2) \rceil$ tree pointers (e.g. more that half full).
The root node has at least two tree pointers unless it is the only node in the tree.

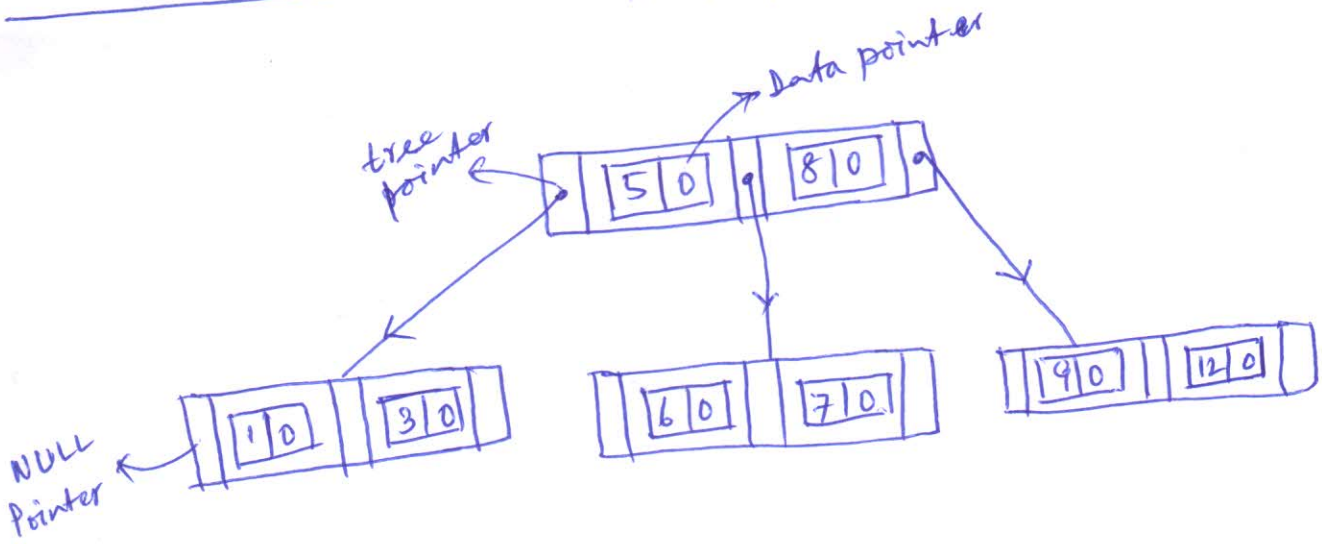
(6) A node with q tree pointers, $q \le p$, has $q-1$ search key field values (and hence has $q-1$ data pointers).

(7) All leaf nodes are at the same level.

A node in a B-tree with q-1 search values



A B-tree of order 3 (The values were inserted in this order 8, 5, 1, 7, 3, 12, 9, 6)



Example (B-tree)

Suppose the search field is $V = 9$ bytes long, the disk block size is $B = 512$ bytes, a record (data) pointer is $P_r = 7$ bytes, and a block (tree) pointer is $P = 6$ bytes.

Each B-tree node can have at most p tree pointers, $p-1$ data pointers and $p-1$ search key field values.

Therefore,

$$(p \times P) + ((p-1) \times (P_r + V)) \leq B$$

$$\Rightarrow 6p + 16(p-1) \leq B$$

$$\Rightarrow 22p \leq 528$$

$$\Rightarrow p \leq 24$$

NOTE: The block usually reserves some space for addressing to the parent pointer, no. of entries in the node. Therefore, the no. of tree pointer p would be (in the above example) 23.

If we assume that each node of the B-tree is 69% full, then $p \times 0.69 = 23 \times 0.69 \approx 16$ tree pointers and 15 search key field values in a node.

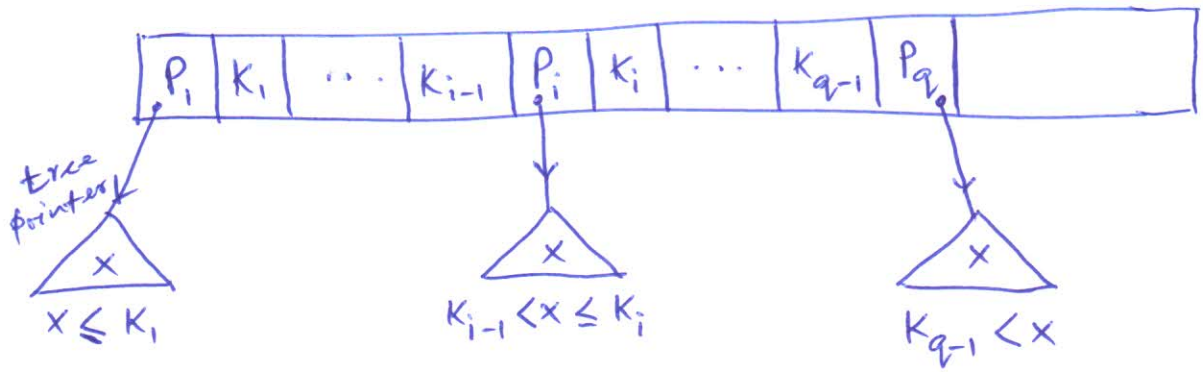
Root :	1 node	15 entries	16 pointers
Level 1 :	16 nodes	240 entries	256 pointers
Level 2 :	256 nodes	3840 entries	4096 pointers
Level 3 :	4096 nodes	61440 entries	

From the above, a two-Level B-tree can hold up to $3840 + 240 + 15 = 4095$ entries on the average; a three-Level B-tree holds up to $61440 + 3840 + 240 + 15 = 65535$ entries on the average.

B^+-trees : Most widely used for dynamic multilevel index.

The structure of internal and leaf nodes of a B^+-tree differs from the structure of a B-tree.

Internal node structure of a B^+-tree



(1) Each internal node is of the form

$$\langle P_1, K_1, P_2, K_2, \cdots, P_{q-1}, K_{q-1}, P_q \rangle$$

where $q \leq p$ and each P_i is a tree pointer.

(2) Within each internal node, $K_1 < K_2 < \cdots < K_{q-1}$

(3) For all search field values X in the subtree pointed at by P_i, we have $K_{i-1} < X \leq K_i$ for $1 < i < q$, $X \leq K_i$ for $i = 1$, and $K_{i-1} < X$ for $i = q$.

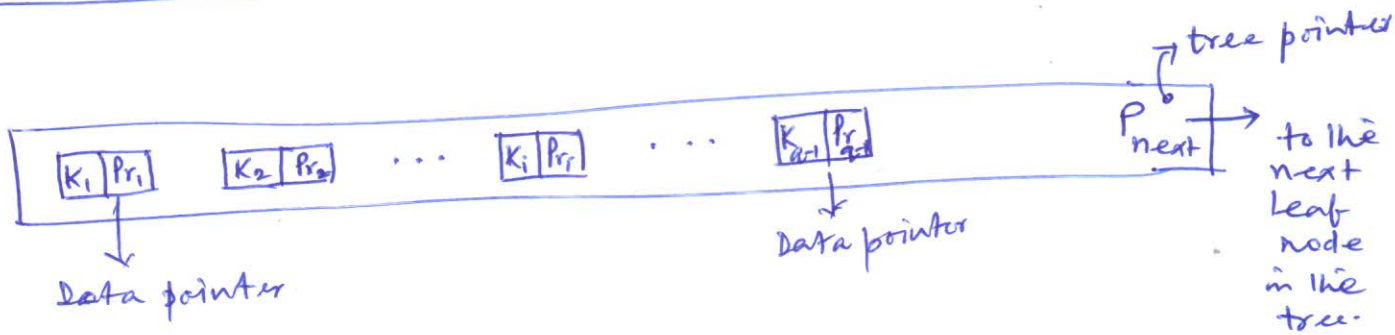
(4) Each internal node has <u>at most</u> p tree pointers.

(5) Each internal node, except the root, has <u>at least</u> $\lceil (p/2) \rceil$ tree pointers. The root node has at least two tree pointers unless it is the only node in the tree.

(6) An internal node with q pointers, $q \leq p$, has $q-1$ search field values.

<u>Note</u> : The data pointers are not stored in the internal nodes.

Leaf node structure of a B+-tree



(1) Each Leaf node is of the form

$$\langle \langle K_1, Pr_1 \rangle, \langle K_2, Pr_2 \rangle, \cdots, \langle K_{q-1}, Pr_{q-1} \rangle, P_{next} \rangle$$

where $q \leq p$, each Pr_i is a data pointer and P_{next} points to the next leaf node of the tree.

(2) within each leaf node, $K_1 < K_2 < \cdots < K_{q-1}$, $q \leq p$.

(3) Each Pr_i is a <u>data pointer</u> that points to the record whose search field value is K_i or to a file block containing the record.

(4) Each leaf node has <u>at least</u> $\lfloor (p/2) \rfloor$ values.

(5) All leaf nodes are at the same level.

<u>NOTE</u>: As the structure of internal node and leaf node in a B+-tree different, the order p of an internal node and p_{leaf} of a leaf node can be different.

Example

Suppose that the search key field is $V = 9$ bytes,
the block size is $B = 512$ bytes,
a record pointer is $P_r = 7$ bytes, and
a block/tree pointer is $P = 6$ bytes.

An internal node can have upto p tree pointers and $p-1$ search field values, and these must fit into a single block. Therefore,

$$(p \times P) + ((p-1) \times V) \leq B$$

$$\Rightarrow (p \times 6) + ((p-1) \times 9) \leq 512$$

$$\Rightarrow 15p \leq 521$$

$$\Rightarrow p \simeq 34$$

[Note: This is much larger than the value of $p = 23$ as in B-tree]

Leaf node (p_{leaf}) in B^+-tree

$$(p_{leaf} \times (P_r + V)) + P_{next} \leq B$$

$$\Rightarrow (p_{leaf} \times (7+9)) + 6 \leq 512$$

$$\Rightarrow 16\, p_{leaf} \leq 506$$

$$\Rightarrow p_{leaf} \simeq 31$$

Therefore, each internal node will have 34×0.69 or approx. 23 pointers and hence 22 key field values.
for leaf node $0.69 \times 31 \approx 21$ data record pointers.

Average no. of entries at each level

Root :	1 node	22 entries	23 pointers
Level 1 :	23 nodes	506 entries	529 pointers
Level 2 :	529 nodes	11638 entries	12167 pointers
Leaf Level :	12167 nodes	255507 entries (record pointers)	

Take away : 255507 record pointers can be managed by a three-level B^+-tree in comparison to 65535 entries for the corresponding B-tree.